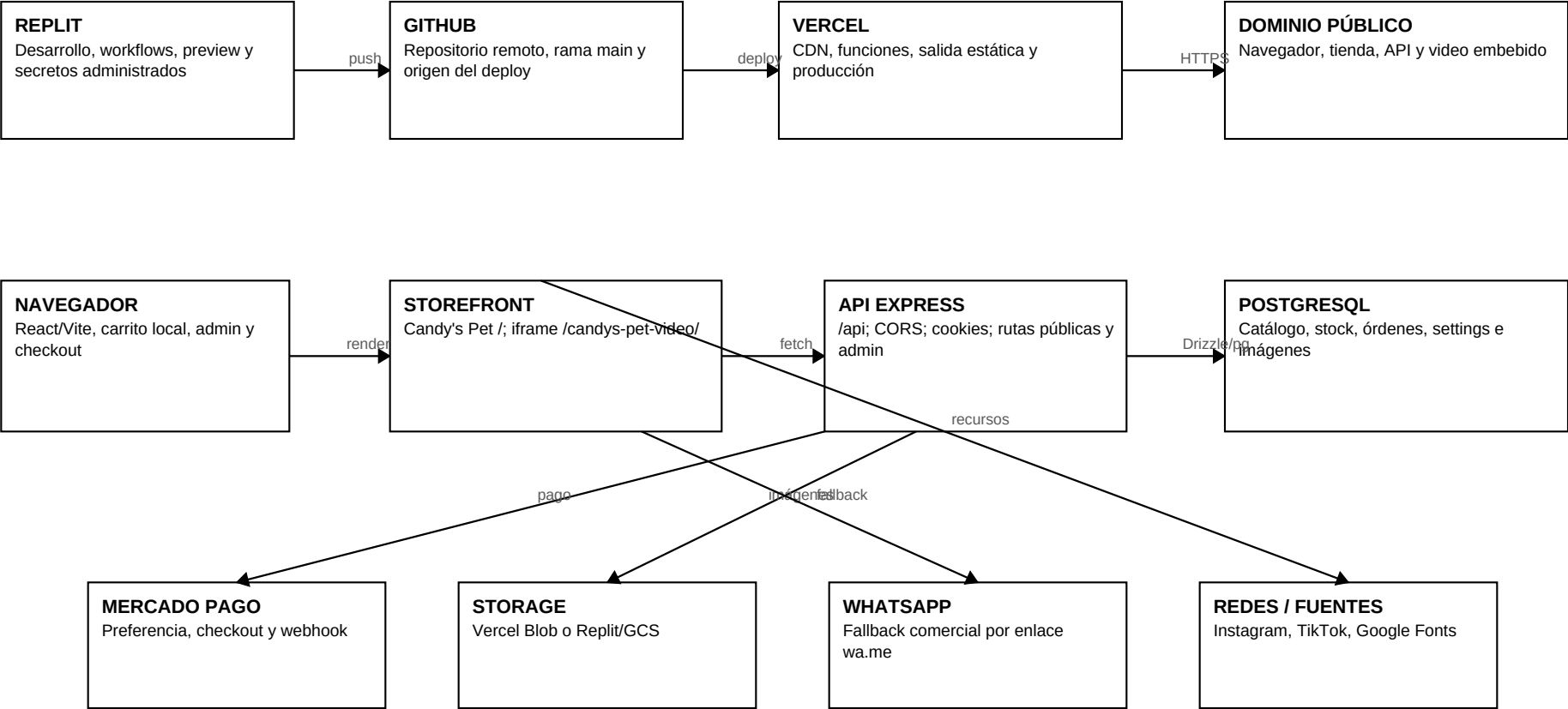


CANDY'S PET

Organigrama técnico de integraciones

Mapa completo de arquitectura, configuración y recorridos de datos. Diseñado para imprimir en blanco y negro y estudiar cómo repetir el patrón en otro proyecto.

ORGANIGRAMA GENERAL



Lectura rápida: el navegador habla con el storefront; el storefront consume el API; el API persiste en PostgreSQL y coordina Mercado Pago y storage. GitHub no procesa pedidos: solo transporta código hacia Vercel. Cloudflare aparece como integración instalada, pero no es parte del flujo de negocio observado. Supabase y Stripe no están activos en el estado actual.

1. Estructura de ejecución y despliegue

Qué corre en Replit, qué se publica en Vercel y cómo viaja una mejora hasta producción.

MONOREPO Y ARTIFACTS

El repositorio es un monorepo pnpm con Node 24 y TypeScript 5.9. La aplicación se separa en cuatro artifacts: API Server (/api, puerto local 8080), Candy's Pet (/ y puerto 25253), candys-pet-video (/candys-pet-video/ y puerto 26272) y mockup-sandbox (/__mockup, puerto 8081). Los workflows de Replit sirven preview y desarrollo; el mockup no aparece en la salida productiva de Vercel.

CADENA DE PUBLICACIÓN



La credencial Git de Replit y el conector GitHub son caminos distintos. Un OAuth válido puede permitir lectura y no permitir escritura; nunca se deben poner tokens en el código.

VERCEL: CONFIGURACIÓN DE BUILD Y ROUTING

vercel.json instala con pnpm install --frozen-lockfile. El build compila el API dos veces (build y build:vercel), compila el video con BASE_PATH=/candys-pet-video/, compila el storefront con BASE_PATH=/ y copia el dist/public del video dentro de artifacts/candys-pet/dist/public/candys-pet-video. El outputDirectory final es artifacts/candys-pet/dist/public. La función api/index.js tiene maxDuration 60 s; el entrypoint carga dinámicamente el bundle ESM precompilado para evitar el choque CJS/ESM.

Componente	Entrada	Salida / ruta	Razón
API	Express + esbuild	api/index.js -> /api/*	Función serverless y bundle ESM
Storefront	Vite + React	dist/public/index.html	SPA pública y fallback
Video	Vite + BASE_PATH	/candys-pet-video/	Iframe estático independiente
Mockup	Vite local	/__mockup	Diseño y preview; no producción

CLOUDFLARE, VERCEL Y LÍMITES DE ALCANCE

Cloudflare y Vercel aparecen como integraciones instaladas del workspace. La arquitectura observada usa Vercel para publicar y no necesita modificar DNS de Cloudflare para el flujo de negocio. No hay workflow GitHub Actions detectado; la automatización documentada es la integración nativa GitHub -> Vercel.

2. Flujo de navegación, API y base de datos

La ruta normal desde una visita hasta catálogo, stock, carrito y consulta de orden.

RECORRIDO PRINCIPAL



QUÉ GUARDA POSTGRESQL

- * stock: producto/talla, nombre, cantidad y timestamp.
 - * settings: claves y valores de precios.
 - * product_images: URL, color y posición de galería.
 - * orders: UUID, preference/payment de Mercado Pago, estado, snapshot JSONB del carrito, total y comprador.
- La conexión usa DATABASE_URL, pg y Drizzle ORM; el esquema se empuja a desarrollo con el comando db push.

CONTRATO DE API Y FALLBACKS

- * GET /api/healthz devuelve {status: ok} y sirve como health check.
- * catalog y stock son públicos por diseño.
- * useCatalog y useStock conservan defaults si el API falla; esto evita una pantalla vacía, pero puede mostrar precios o disponibilidad stale.
- * El carrito vive en el browser: productId, talla, color, precio y cantidad. No existe orden hasta iniciar el checkout.
- * GET /api/payment/order/:orderId existe, pero PaymentResult no lo usa: la pantalla de retorno se basa en query params.

LECCIÓN REUSABLE

Separar lectura pública, mutaciones autenticadas y persistencia ayuda a razonar el sistema. El frontend puede optimizar UX, pero el API debe volver a validar stock, precio, identidad de la orden y estados antes de escribir. Una respuesta 200 no significa que una mutación haya terminado: la UI debe actualizar o consultar el resultado real después de navegar o recargar.

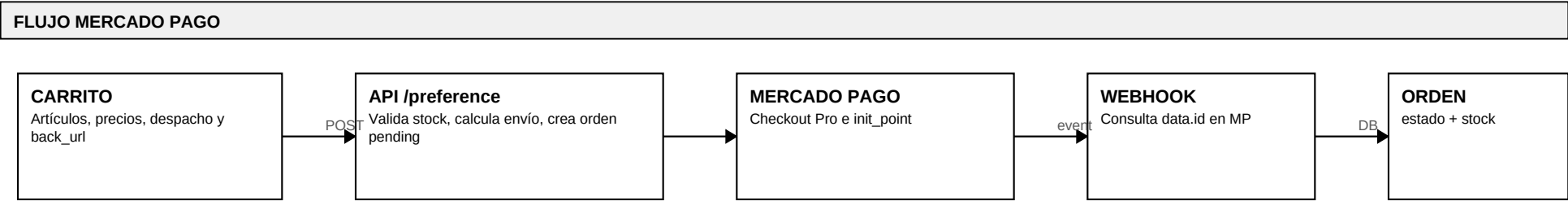
Capa	Responsabilidad	Entrada	Salida
Browser	Interacción y estado temporal	Clicks, localStorage, query	fetch, redirect, links
Express	Reglas, auth y orquestación	JSON, cookies, webhook	JSON, redirect, DB writes
PostgreSQL	Fuente de verdad	Drizzle queries	rows, order state, stock

Documento privado de aprendizaje. No incluye valores de secretos, tokens ni credenciales.

3

3. Mercado Pago y caminos de compra

Checkout Pro, webhook, estados de orden y fallback comercial.



CONFIGURACIÓN Y ESTADOS

La dependencia mercadopago crea preferencias con MP_ACCESS_TOKEN en el servidor. Sin esa variable, la API responde 503 MP_NOT_CONFIGURED y el frontend deriva a WhatsApp. El frontend redirige a init_point y vuelve con payment=success[failure]pending. El webhook consulta el pago directamente en Mercado Pago, usa external_reference como orderId, guarda payer/status/payment y descuenta stock cuando el pago es aprobado. La orden evita repetir el descuento si llega otra aprobación.

FALLBACKS Y LO QUE NO ESTÁ IMPLEMENTADO

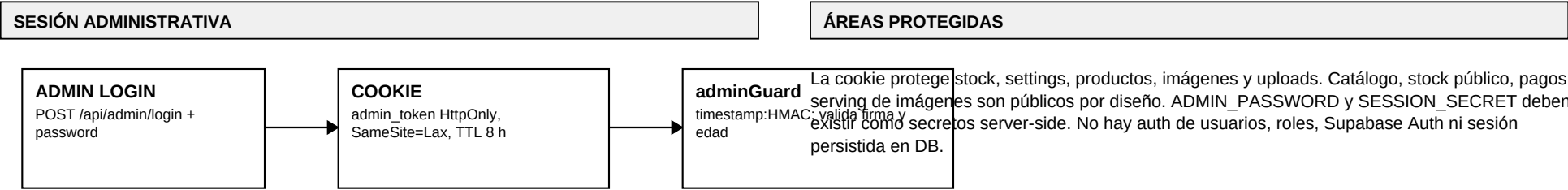
Transferencia bancaria, retiro presencial, consultas y pedidos personalizados abren enlaces wa.me con el texto del carrito. WhatsApp no tiene API, webhook ni almacenamiento en el backend. MP_PUBLIC_KEY aparece como secreto disponible del entorno, pero no se observó uso en el código actual; el checkout usa el token de servidor. No hay Stripe activo: solo aparece en historial como integración antigua.

CUIDADO ANTES DE REUTILIZAR

Punto	Estado observado	Mejor práctica para otro proyecto
Precio	Productos normales conservan unit_price recibido del navegador; carnet normaliza formatos.	Derivar precio desde DB o catálogo firmado en servidor.
Origen	back_url del cliente se reutiliza en retorno y notification_url.	Usar dominio de allowlist y construir URLs server-side.
Stock	Verificación previa; updates separados; puede haber sobreventa concurrente.	Transacción con lock/UPDATE atómico y resultado idempotente.
Webhook	Responde 200 incluso en errores y no verifica firma explícita.	Validar firma, devolver error en fallas y mantener idempotencia.

4. Administración, sesión y seguridad

Autenticación propia para operaciones internas; no hay cuentas de clientes ni SSO.



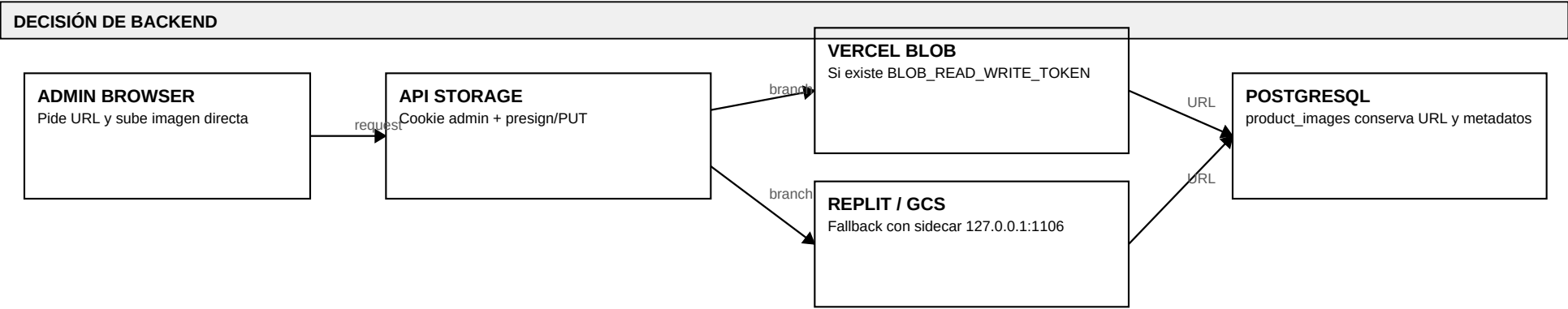
CONFIGURACIÓN SENSIBLE (NOMBRES, NUNCA VALORES)		
Nombre	Uso	Qué debe ocurrir
DATABASE_URL	Conexión PostgreSQL/Drizzle	Obligatoria; gestionar con secret manager.
ADMIN_PASSWORD	Contraseña de admin	Nunca dejar fallback en producción; rotar.
SESSION_SECRET	Firma HMAC de cookie	Secreto largo; no usar dev-secret.
MP_ACCESS_TOKEN	Mercado Pago server-side	No exponer al bundle; rotar y limitar.
BLOB_READ_WRITE_TOKEN	Vercel Blob	Solo API; seleccionar storage de forma explícita.
PRIVATE_OBJECT_DIR / PUBLIC_OBJECT_SEARCH_PATHS	Storage Replit/GCS	Separar objetos privados y públicos.
PORT / BASE_PATH / NODE_ENV / LOG_LEVEL / REPL_ID / CI	Runtime y build	No son credenciales; cambian ejecución.
SUPABASE_DATABASE_URL / MP_PUBLIC_KEY	Disponibles en entorno	No se detectó uso activo en el código actual.

RIESGOS A CORREGIR AL ENSEÑAR ESTE PATRÓN

El código contiene fallbacks de desarrollo para password y secreto si faltan variables; la comparación no usa timingSafeEqual y no hay rate limiting. CORS refleja cualquier origen con credentials:true. En un proyecto nuevo, fallar al arrancar si falta un secreto es más seguro que continuar silenciosamente.

5. Imágenes y almacenamiento de objetos

Dos backends posibles; la selección se hace por configuración y cambia el recorrido.



DETALLES DE FUNCIONAMIENTO

- * Vercel Blob: /api/storage/uploads/direct recibe hasta 10 MB, escribe objeto público y sirve por redirect/list.
- * Replit/GCS: PUBLIC_OBJECT_SEARCH_PATHS localiza públicos; PRIVATE_OBJECT_DIR permite URLs firmadas de PUT por 15 minutos.
- * La DB guarda URL/metadatos, no los bytes.
- * Los endpoints de serving son públicos; el upload requiere cookie admin.
- * Al borrar fila no hay limpieza automática garantizada del objeto: pueden quedar blobs huérfanos.

PRIVACIDAD Y DISEÑO

Las imágenes de productos son públicas intencionalmente. Las fotos de clientes pueden contener teléfonos, nombres o datos de contacto: deben sanitizarse antes de galería, testimonio o asset público. Replit Object Storage depende del sidecar de Replit; una URL firmada no es un reemplazo universal para una política de acceso. En Vercel, usar Blob exige que la variable exista y que el flujo quede probado en ese entorno.

REGLA REUSABLE

Separar bytes de metadatos permite cambiar proveedor sin migrar órdenes ni productos: el API controla autorización y metadatos; el storage maneja bytes; PostgreSQL conserva la referencia consultable. Para migrar, primero define quién puede leer, quién puede escribir, cuánto dura una URL firmada y cómo se borran objetos huérfanos.

6. Vídeo, enlaces y servicios sin API

Diferenciar una integración operativa de un enlace externo o un recurso cargado por el navegador.

ARTIFACT DE VÍDEO

candys-pet-video es un artifact web independiente. Vite usa `BASE_PATH=/candys-pet-video/` para que sus assets relativos no escapen del subpath. El storefront lo monta en un `iframe` con `allow=autoplay`. El video `cozy-puppy.mp4` aparece en escenas; `bg_music.mp3` acompaña la plantilla de cinco escenas de aproximadamente ocho segundos. `VideoSection` y `VideoTemplate` intercambian `candys:audio-focus` por `postMessage` para atenuar audio según el foco. Es un flujo estático, no API ni Supabase.

OTROS RECURSOS VISUALES

El carnet veterinario tiene además un MP4 importado desde `attached_assets` y reproducido inline. Ese archivo y los exports descargables no son la fuente del `iframe` principal. Google Fonts se carga desde `fonts.googleapis.com` y `fonts.gstatic.com` en storefront, video y mockup: no hay self-hosting y el navegador contacta a Google.

CLASIFICACIÓN DE INTEGRACIONES EXTERNAS			
Servicio / marca	Qué hace aquí	Tipo de integración	Qué no hace
WhatsApp / wa.me	Fallback de pago, retiro, consultas y pedidos	Link externo con texto codificado	No hay API, webhook ni historial local
Instagram / TikTok	Links de marca, colaboración y tráfico	Link estático	No hay SDK, login, pixel ni sincronización
Google Fonts	Tipografías de navegador	Carga HTTP desde CDN externo	No es auth ni almacenamiento
Blue Express / Starken / Chilexpress	Nombre/logo y tarifa fija seleccionada	Dato de pedido + presentación	No hay API, tracking ni etiquetas
Cloudflare	Integración instalada del workspace	Infra de cuenta, no flujo observado	No se cambió DNS para esta app
Supabase	No implementado	Solo nombre de secret disponible	No hay cliente, tablas ni Auth
Stripe	Antecedente histórico	Commits antiguos	No hay dependencia ni checkout vigente
CÓMO LEER ESTA DISTINCIÓN			

Si no existe SDK, endpoint, webhook, token usado o sincronización, no conviene llamarlo integración operativa. Documentarlo como enlace, recurso CDN, dato visual o legado evita prometer capacidades que el sistema no tiene.

7. Matriz completa de integraciones

Inventario consolidado: propósito, dirección de datos, autenticación y estado.

Integración	Dirección de datos	Autenticación / config	Estado actual
Replit	Código y navegador -> workflows; API -> sidecar storage	Secrets, workflows, Node 24, DB provisionada	Activo para desarrollo/preview
GitHub	Replit commit -> main -> Vercel	Git remoto/conector; no mostrar tokens	Origen de despliegue; permisos deben verificarse
Vercel	Git -> build -> CDN/Function -> navegador	Proyecto + vercel.json; Blob token opcional	Producción: API, tienda y video
PostgreSQL + Drizzle	API <-> rows de catálogo, stock, órdenes	DATABASE_URL	Fuente de verdad de negocio
Mercado Pago	API -> preference; MP -> webhook -> API/DB	MP_ACCESS_TOKEN	Checkout condicional activo
Vercel Blob	API/admin -> bytes públicos; DB -> URL	BLOB_READ_WRITE_TOKEN	Branch si token existe
Replit Object Storage/GCS	API/admin <-> sidecar <-> objetos	PRIVATE_OBJECT_DIR; PUBLIC_OBJECT_SEARCH_PATHS	Fallback en runtime Replit
Admin auth propia	Browser -> login -> cookie -> API admin	ADMIN_PASSWORD; SESSION_SECRET	Activo; requiere hardening
WhatsApp	Browser -> wa.me con carrito	Cuenta del usuario; sin API	Fallback comercial
Instagram/TikTok	Browser -> link externo	Ninguna	Enlaces de marca
Google Fonts	Browser -> fonts.googleapis.com/gstatic	Ninguna	CDN tipográfico
Transportistas	Browser -> API como selección de envío	Ninguna	Presentación; sin API logística
Cloudflare	No hay flujo de negocio observado	Cuenta instalada	No usar como evidencia de DNS cambiado
Supabase	No hay flujo de código	SUPABASE_DATABASE_URL disponible, no usado	No activo
Stripe	No hay flujo actual	Historial solamente	Legado no operativo

VARIABLES Y SECRETOS: INVENTARIO SEGURO

Nombres detectados/disponibles: DATABASE_URL, MP_ACCESS_TOKEN, MP_PUBLIC_KEY, SESSION_SECRET, ADMIN_PASSWORD, BLOB_READ_WRITE_TOKEN, PRIVATE_OBJECT_DIR, PUBLIC_OBJECT_SEARCH_PATHS, DEFAULT_OBJECT_STORAGE_BUCKET_ID, SUPABASE_DATABASE_URL, PORT, BASE_PATH, NODE_ENV, LOG_LEVEL, REPL_ID y CI. Este PDF deliberadamente no contiene valores. La regla general es inyectar credenciales en runtime, validarlas al iniciar y nunca compilar secretos en el frontend.

8. Guía para repetir la arquitectura

Un método práctico para construir otro proyecto sin perder trazabilidad.

PASO A PASO REUSABLE

- * **1. Define límites**
Separa storefront, API, DB, media y deploy. Decide qué es público y qué requiere sesión.
- * **2. Define contratos**
Escribe OpenAPI/Zod: entradas, salidas, errores, estados y quién deriva cada campo.
- * **3. Separa secretos**
Usa secret manager; el navegador recibe solo valores públicos. Falla al iniciar si falta lo crítico.
- * **4. Diseña la persistencia**
Guarda metadatos en DB y bytes en object storage; registra ownership, TTL y limpieza.
- * **5. Haz el checkout server-side**
El navegador propone; el API valida precio, stock, total, origen y genera la orden.
- * **6. Haz webhooks idempotentes**
Consulta al proveedor, valida firma, guarda evento, actualiza estado y responde con el código correcto.
- * **7. Define deploy reproducible**
Un comando frozen-lockfile, output único, rutas API explícitas y smoke tests de salud.
- * **8. Audita integraciones**
Para cada servicio escribe propósito, dirección, auth, estado y lo que no hace.

CHECKLIST DE VERIFICACIÓN

- Antes de publicar:
- ☐ GET /api/healthz responde 200.
 - ☐ Catálogo, stock, imágenes y rutas SPA cargan en el dominio real.
 - ☐ Checkout sin token falla de forma explícita y ofrece fallback.
 - ☐ Webhook prueba approved, rejected, pending y duplicado.
 - ☐ Precio y stock se calculan en servidor.
 - ☐ Cookie admin expira, no usa defaults y tiene CSRF/origin control.
 - ☐ Storage valida tamaño, ownership y limpieza.
 - ☐ Video y assets respetan BASE_PATH y no devuelven index.html por error.
 - ☐ No se imprimen secrets en logs ni PDF.
 - ☐ Se verifica el deployment READY y un smoke test público.

PRIORIDADES TÉCNICAS DETECTADAS

1) Eliminar fallbacks de credenciales de desarrollo. 2) Restringir CORS y derivar URLs desde una allowlist. 3) Validar precios contra catálogo/DB. 4) Hacer stock atómico. 5) Validar firma y reintentos de webhook. 6) Hacer que PaymentResult consulte la orden real. 7) Completar los datos de mascota del carnet. 8) Proteger datos de clientes y limpiar blobs huérfanos. 9) Mantener separado el artifact de video y probar sus rutas.

9. Fuentes auditadas y glosario

Puntos de entrada para leer el código y verificar cada afirmación.

FUENTES PRINCIPALES

Configuración: vercel.json, .replit, replit.md, artifact.toml de cada artifact.
API: artifacts/api-server/src/app.ts, src/index.ts, routes/catalog.ts, stock.ts, payment.ts, admin.ts, storage.ts.
Persistencia: lib/db/src/index.ts, drizzle.config.ts y lib/db/src/schema/*.
Frontend: artifacts/candys-pet/src/hooks, context/CartContext.tsx, components/CartDrawer.tsx, pages/AdminPage.tsx, components/VideoSection.tsx y pages/PaymentResult.tsx.
Video: artifacts/candys-pet-video/src/components/video/VideoTemplate.tsx y video_scenes/*.
Memoria operativa: .agents/memory/vercel-github-auto-deploy.md, vercel-monorepo-api-entry.md, customer-image-privacy.md, imported-artifact-registration.md y github-push-auth.md.

GLOSARIO

Artifact: unidad ejecutable/preview registrada en el workspace.
SPA: aplicación de una sola página; Vercel sirve index.html para rutas del frontend.
Function: backend serverless invocado por /api.
Webhook: llamada de un proveedor al backend para notificar un evento.
Presigned URL: URL temporal para subir/leer un objeto sin exponer credenciales.
Idempotencia: repetir un evento no repite sus efectos.
Fallback: comportamiento alternativo cuando un servicio no está configurado o falla.
Origin: dominio que inicia una petición; debe controlarse en CORS y URLs de retorno.
Snapshot: copia JSON del carrito guardada para conservar qué se compró.

NOTA FINAL

Este documento describe el estado observado del proyecto y distingue explícitamente lo activo, lo opcional, lo histórico y lo que solo es un enlace. Para aprender, siga las flechas de cada diagrama y luego abra las fuentes indicadas: primero el contrato API, después la ruta del navegador y finalmente la integración externa. La arquitectura más importante no es el proveedor; es la dirección de los datos, la autoridad de cada campo y la forma de confirmar que una operación terminó realmente.

Privacidad: este PDF se genera como archivo descargable privado. No se publica en la página de Candy's Pet. No contiene contraseñas, tokens, connection strings ni valores de secretos.